ملاحظات مبعثرة من الكورس 6

الملاحظات المكتوبة قليلها شرحه الدكتور في فيديوهاته وأكثرها إجتهاد شخصي.

في أغلب الملف عند كتابة كلام بين قوسين رياضيات [ ] فهذه مسميات اختيارية أو شرح.
ويفضل حذف المكتوب مع الأقواس [ ] عند البرمجة.

أوامر تنظيف ال cin اذا كان المدخل خاطئ :

cin.clear() تزيل حالة الخطأ لأن ال cin تدخل في حالة خطأ عند شيئ مخالف لما يتوقع إدخاله. فإن الكود cin.clear() يقوم بإخراج ال cin من حالة الخطأ ويجعلها تسمح باستقبال جديد .

أما cin.ignore فيجعل ال buffer (المكان الذي يخزن مدخل ال cin بشكل شبه مؤقت) يتجاهل كل المدخل حتاما ضغط المستخدم على زر enter (/n) .

هذا هو إختلاف المهام بين الكودين .

أما كود ال ignore ... الذي هو...

```
std::numeric_limits<std::streamsize>::max(), '\n'
```

فإن ال numeric_limits هي عبارة عن كلاس فيه عضو يستطيع معرفة القيمة العظمى وهذا الكلاس فيه عدة أدوات مثل إعطاء أصغر قيمة أو دعم الأعداد السالبة إلخ... يقوم بمعرفة القيمة العظمى لأي نوع تعريف ...
(int... float... bool... streamsize... etc...)
و streamsize هو نوع بيانات يمثل عدد خانات الإدخال أو الإخراج .
و max تمثل أعظم قيمة للنوع المختار . وهي أداة من numeric_limits .

الكود كله مع بعضه أصبحت ترجمته ...

تعريف كلاس std::numeric_limits يقوم بأعطائي كل الخانات المدخلة <std::streamsize> باستخدام الأداة max حتاما نصل الى ضغطة enter (\n) .

طبعا وضعنا أقواس لل max لأنها عبارة عن دالة وأي دالة في لغة C++ يجب أن يوضع لها أقواس .

التعامل مع البارامیترات الإفتراضية القيم في دالة declaration :

لدينا المثال ...

```
#include<iostream>
#include<string>
using namespace std;

int Sum(int, int, int=0, int=0);


int main()
{
	int n1=10, n2=20, n3, n4;
	cout << Sum(n1, n2) << endl;
}

int Sum(int n1, int n2, int n3, int n4)
{
	return (n1 + n2 + n3 + n4);
}
```

عندما أعطينا قيم افتراضية أصفار فإننا أعطيناها للبارامیترات المعرفة في ال declaration وليس في ال definition لأن البرنامج بعد استدعاء الدالة يذهب الى ال declaration أولا الذي يحوله الى ال definition ولذلك يجب أن تكون القيم الإفتراضية معطاة لبارامیترات ال declaration .

الطباعة التنسيقية في لغة C++ :

سطر الكود printf يعني print formatted (طباعة منسقة) .

الإشارة % تعني إبدأ التنسيق .

الإشارة * سوف يتم إستبدالها بعدد الخانات المطلوبة .

إذا وضع 0 قبل الإشارة أي أن الخانات المطلوب بعددها مهما كان. أريدها أن تكون خانات صفرية (من أصفار) .

إذا وضع إشارة الفاصلة (.) معناها أريد عدد فواصل معين بعد آخر خانة في الرقم (عدد الفواصل أنا أحدده بنهاية سطر الطباعة التنسيقية) .

تأثير الإشارة & :

عندما نضيف الإشارة الى تعريف متغير ما... فإننا في ذلك الوقت لا نعرف متغير جديد بل نعرف تعبير للمتغير الذي سيمرر لهذا التعبير... فعندما نعدل على التعبير نحن نعدل على المتغير المعبر عنه بالتعبير (نحن نعرف اسم لمتغير وليس متغير جديد).
ومن هنا جاء المسمى reference (مرجع) لانها دلالة تعريف مرجع لمتغير آخر وليس متغير جديد.
ما يعرف على أنه مرجع مثل المرآة المنحنية... أي تعديلات ترسلها للمرآة فهي تعكسها الى المتغير الذي عرفتها على انها مرآة منحنية لأجله.

التعامل مع أمر الادخال cin في ال buffer :

- ال cin تتجاهل المحرف الدال على السطر الجديد (زر enter الذي ضغطته بعد عملية الادخال) والفراغات الظاهرة في بداية الاستكشاف في ال buffer...
- تتوقف عن الابتلاع في اللحظة التي تشاهد فيها المحرف الدال على السطر الجديد (زر enter الذي ضغطته بعد عملية الادخال) أو فراغ.
- تبقي المحرف الدال على السطر الجديد (زر enter الذي ضغطته بعد عملية الادخال) و الفراغ في ال buffer ولا تجعل ال buffer يتجاهله... (لا تستهلكه)

التعامل مع أمر الادخال getline في ال buffer :

- ال getline لا تتجاهل المحرف الدال على السطر الجديد (زر enter الذي ضغطته بعد عملية الادخال) والفراغات الظاهرة في بداية الاستكشاف في ال buffer...
- تتوقف عن الابتلاع في اللحظة التي تشاهد فيها المحرف الدال على السطر الجديد (زر enter الذي ضغطته بعد عملية الادخال) أو فراغ.
- لا تبقي المحرف الدال على السطر الجديد (زر enter الذي ضغطته بعد عملية الادخال) و الفراغ في ال buffer و تجعل ال buffer يتجاهله... (تستهلكه).

اختصاص إشارة ال دوت (.) في لغة C++ :

بشكل عام هي إشارة توجيه إلى بيانات... مثلا عندما نعرف هيكل ما ونخزن فيه قالب متغيرات...
فاذا اردنا الوصول لمتغيرات هذا الهيكل نستخدم إشارة ال دوت بعد كتابة اسم الهيكل المستنسخ من هذا الهيكل ونرى اننا نستطيع كتابة أسماء المتغيرات داخل هذا المتغير... أي أن ال دوت تشير الى المتغير الذي سوف اكتب اسمه بعدها...
مثلا عند استخدام الأمر...
[name of the vector].push_back()
ما يحدث أن إشارة الدوت بعد كتابتها نستطيع الوصول لكافة الدوال الموجودة داخل مكتبة تعريف الفيكتور المكتوب اسمه قبل الدوت.
و push_back هي دالة من هذه الدوال (تقوم بإرجاع قيمة الى آخر خانة في الفيكتور)

تأثير استخدام الأمر pop_back() على الفيكتور :

عند عمل pop_back() الكمبيوتر لا يغير المساحة المحجوزة للفيكتور في الذاكرة ram (سعة الفيكتور في الرام) ولكن يصغر حجم الفيكتور ويتقلص على حسب عدد العناصر الموجودة فيه.
السعة المحجوزة... capacity
الحجم... size
الأمر cout << [name of the victor].size يعرض حجم الفكتور الحالي.
الأمر cout << [name of the victor].capacity يعرض السعة المحجوزة للفكتور لحظة تنفيذ هذا الكود.
ولو عملنا عدة pop_up ومن ثم أردنا جعل السعة تتساوى مع حجم الفيكتور (السعة تنقص للتساوى مع الحجم) نستخدم الأمر...
[name of the vector].shrink_to_fit()

استخدام {} عند تعريف المتغيرات :

دائما عند تعريف متغير لا يحتوي على قيمة حاليا... قم باستخدام الأقواس الكيرلي بجانبه... {}
مثال على تعريف متغير بدون قيمة حالية مع وبدون ال {} :
int number; يعطى له قيمة عشوائية.
string name; يعطى له قيمة خالية تلقائيا لأنه كائني وله باني (سنتعلم معنى ذلك بمرحلة لاحقة أنا نفسي لا أعرف معنى ذلك) ولكن لا ضرر إذا ما وضعنا الأقواس المتعرجة بجانبه.

int number{}; الآن أنت تعطيه قيمة فراغ حالية (صفر).

تحديد أين يشير المؤشر في عناصر الفيكتور :

الأمر: [name of the vector].begin() يقوم بتحديد المؤشر على أول عنصر في الفكتور.
اذا ما أردنا تحديد العنصر الثالث مثلا (عنوانه 2) : [name of the vector].begin() + 2

حذف عنصر محدد من عناصر الفيكتور باستخدام الدالة erace() :

باستخدام [name of the vector].erace([member address])
مثال يبين طريقة استخدام ال erace والفرق بينها وبين ال pop_back وطريقة حذف مجال عناصر :

◆ مقارنة بين erase() و pop_back()

الخاصية	pop_back()	erase()
ما تحذف؟	العنصر الأخير فقط	عنصر واحد أو نطاق عناصر محدد
طريقة الاستخدام	v.pop_back();	v.erase(v.begin() + index); أو v.erase(v.begin()+start, v.begin()+end);
زمن التنفيذ	ثابت $O(1)$	خطي $O(n)$ بسبب إعادة ترتيب العناصر
القيمة المرجعة	لا تُرجع شيئًا	تُرجع iterator يشير للعنصر التالي بعد المحذوف
الحفاظ على الترتيب	يحافظ على الترتيب طبيعيًا	يحافظ على الترتيب، لكن قد يكون مكلفًا
الاستخدام الأمثل	عند الحاجة لحذف آخر عنصر بسرعة	عند الحاجة لحذف عنصر أو مجموعة عناصر من أي مكان

Index هي عنوان العنصر المراد حذفه.

ملاحظة : كلا الدالتين لا تقومان بتقليل السعة... إنما الحجم فقط.

ملاحظة على الأمر shrink_to_fit() :

لا تستخدم هذه الأمر اذا ما كان من الممكن أن يعاد إضافة عناصر الى الفيكتور...
لأن عملية إعادة الحجز مكلفة وتسبب بطأ البرنامج.
استخدمها فقط اذا كنت ضامن أن البرنامج لن يعيد إضافة عناصر الى الفيكتور الذي قلص حجمه.

تأثير الدالتين front() و back() في الفيكتور :

[name of the vector].front()
[name of the vector].back()

ال front() تعيد مرجع للعنصر الأول في الفيكتور وكأنها تعيد مسمى لهذا العنصر
(كما تعلمنا في إشارة ال & التي تدخل لتعريف مرجع).
ال back() تعيد مرجع للعنصر الأخير في الفيكتور وكأنها تعيد مسمى لهذا العنصر
(كما تعلمنا في إشارة ال & التي تدخل لتعريف مرجع).

الدالة clear() في مكتبة ال vector :

اذا استخدمنا الأمر [name of vector].clear(); فهي تقوم بمسح كافة عناصر الفيكتور ولكن هي تمسح الحجم ولكن لا تمسح السعة.

تنفيذ أوامر تتضارب مع حالة الفكتور :

إذاما قمنا بطلب دالة front() أو back() والفيكتور فارغ سوف يحدث error لأن الفكتور لا يحتوي على أي عناصر وانت تطلب العنصر الأول أو الأخير... كيف ذلك؟!!!

آلية عمل ال pointer المعرف (int , float , char ...) وبعض الملاحظات عليه :

جزء 1 :

ال pointer هو خزان مثل أي متغير آخر ولكنه خزان من نوع تخزين عناوين (أي أن ما يخزن فيه هو عناوين وليس قيم) عناوين متغيرات أخرى مهما كان نوعها (متغير صحيح... متغير عشري... مصفوفة... هيكل... الخ...).

مثال على طريقة تخزين عنوان متغير آخر ضمن بوينتر...

```
int a
int *p=&a
```

اذا اردنا طباعة عنوان متغير a بالاعتماد على البوينتر نكتب الكود...

```
cout << p;
```

واذا اردنا طباعة قيمة a (بفرض أننا خزنا فيها قيمة) باستخدام البوينتر...

```
cout << *p;
```

النجمة تخيلها أنها المفتاح الذي يفتح باب خزان المتغير الذي يشير اليه البوينتر.
أي تعديل على p* يعدل على a
بدون المفتاح (فقط p) أنت تستطيع الولوج الى عنوان الخزان ولكن ليس القيمة المخزنة في الخزان.

إذا أردنا تعريف بونتر يشير الى عنوان البوينتر p* نعرفه بالشكل... (pp اسم اختياري)

```
int **pp=&p;
```

الآن نستطيع الوصول الى قيمة المتغير a عن طريق البوينتر pp

وكلما ارتفعنا بتعريفات البوينترات كلما زدنا عدد النجوم (عدد المفاتيح).

جزء 2 :

عند تعريف مصفوفة وتريد الإشارة الى عنوانها باستخدام البوينتر. نكتب الكود...

```
int arr[4] = {2,1,4,3};
int(*p)[4] = &arr;
```

دلالة أن البوينتر هذا يشير الى مصفوفة من 4 عناصر.

إذا أردت الإشارة الى العنصر في المصفوفة أستطيع كتابة الكود...

```
int* p = arr;
```

عندما أكتب arr فأنا أشير بالفطرة الى عنوان أول عنصر في المصفوفة لأن المصفوفة هي بحد ذاتها مؤشر يشير الى عنوان أول عنصر فيها اذا لم أحدد أين تشير.

أو أستطيع كتابة الكود الذي هو نفسه الكود السابق وستفهم لماذا اذاما فهمت آخر سطرين شرح كتبتهم...

```
int *p = &arr[0];
```

في هذا الكود...

```
int arr[2] = { 1,2 };
int* p = arr;
cout << p << endl;
p++;
```

عندما كتبت p++ فأنا لا أقول زد على العنوان بمقدار واحد... انما أقول للمؤشر أن يقفز بمقدار حجم واحد من نوع int (يقفز فوق العنصر الأول)

كل متغير من نوع int في برنامج vs com يحجز مساحة في الذاكرة وقدرها 4 بايت.

في الكود السابق أنا أخبر البوينتر أن يقفز فوق حجم int واحد فقط.

وإذا أخبرته أن ينتقل الى العنوان 3 مثلا (العنصر الرابع) فيجب أن يقفز فوق 3 أحجام وصولا الى الحجم الرابع (العنصر الرابع) (يجب أن يقفز فوق مجمل حجم 12 بايت).

آلية عمل ال pointer المعرف على أنه **عام** وبعض الملاحظات عليه :

عند تعريف بوينتر على أنه void فإننا نقول : أن هذا المؤشر معرف على أنه عام ويمكنه تخزين عنوان أي متغير مهما كان نوع هذا المتغير. ولكن لا يمكن لهذا ال pointer الوصول الى قيمة هذا المتغير. **لذلك نقوم باستخدام الكود...**

```
cout << *(static_cast<[type of the variable you gave the pointer its address]*>([name of the pointer])) << endl;
```

مثال حقيقي على ذلك...

```
int a = 20;
void* p = &a;
cout << *(static_cast<int*>(p)) << endl;
```

هذه هي الطريقة لطباعة قيمة المتغير المخزن عنوانه في البوينتر.

لماذا لا يمكن لل pointer الوصول الى قيمة هذا المتغير؟...
لأنه يستطيع تخزين العنوان ولكن لا يستطيع معرفة نوع القيمة المخزنة في هذا المتغير المعنون بهذا العنوان لذلك لن يستطيع الولوج الى خزان هذا المتغير (قيمته) .

حيث أن ال pointer يعرف العنوان فقط ولكن لا يعرف كم الحجم من الذاكرة الذي يجب أن يقرأ منه أو يكتب اليه (اذا نوع القيمة int ... فالحجم 4 بايت) اذا البوينتر عرف الحجم سيستطيع الولوج اليه.

لماذا سمي الأمر static_cast بهذا الاسم؟

سُمّيت بـ static_cast لأنها تعتمد على **الفحص الثابت (compile-time checking)** أثناء التحويل بين الأنواع، أي أن المترجم يقرر صحة التحويل أثناء الترجمة، وليس في وقت التشغيل. كلمة *static* هنا تشير إلى أن القرار يتم بشكل "ثابت" عند الترجمة، وليس ديناميكيًا مثل dynamic_cast [1] [2] .

ولماذا إستخدمت الأقواس < > وليس الأقواس التي اعتدنا عليها ( ) في الكود؟...

لأن هذه الأقواس تميز نوع عملية التحويل (static) وليس أي نوع عملية تحويل أخرى بالإضافة ليحدث اتساق باللغة لأن هذه الأقواس دائما تستخدم للتعريف في لغة ++c مثلا في تعريف الفيكتور...

```
vector <int> v;
```

المساحة المحجوزة في الذاكرة بشكل مؤقت بالتعاون مع ال pointer :

المساحة المؤقتة مهمتها إدارة الذاكرة قدر المستطاع حيق يمكن حجزها و حذفها أثناء تشغيل المشروع (أثناء ال run time) أما لو استخدمنا الطريقة التقليدية فسوف نحجز مساحة في الذاكرة لمتغير وسوف تبقى هذه المساحة محجوزة حتى بعد إنتهاء مهمتها وذلك يسبب بطئ في البرنامج.

كيف أحجز هذه المساحة المؤقتة؟
هذا المثال سوف يشرح...

```
int* pntr1;
pntr1 = new int;
*pntr1 = 20;

cout << pntr1 << endl;
```

ما حدث هنا أننا عرفنا pointer
ثم عرفنا في الذاكرة مساحة ل int طبعا أستطيع التعريف من أي نوع أريد...
ما حصل في سطر الكود الثاني هو أننا حجزنا في الذاكرة مساحة وأعطينا عنوان هذه المساحة إلى ال pointer
الآن اذا أعطيت لل pointer قيمة ليوجهها الى المساحة المحجوزة في الذاكرة التي أعطينا عنوانها لى ال pointer سوف يقوم بتخزين هذه القيمة في هذه المساحة المحجوزة كما حدث في السطر 3 من الكود.
ملاحظة : الدالة new هي من تخزن عنوان المساحة المحجوزة في البوينتر. (نتحدث في السطر الثاني)

وفي النهاية طبعنا البيانات المخزنة في المساحة المحجوزة.

الآن أستطيع حذف هذه المساحة من الذاكرة لأنه لم يعد لي حاجة لها. وهنا تكمن فائدة هذه الطريقة من التعريف حيث أنني أستطيع مسح المساحة المحجوزة للقيمة لاحقا... ولكن لو أنني عرفت متغير وخزنت به هذه القيمة لن أستطيع إدارة الذاكرة وسوف تبقى هذه المساحة محجوزة إلى نهاية البرنامج وسيسبب هذا بطئ في البرنامج.

كيف أحذف المساحة المحجوزة في الذاكرة؟ **باستخدام الكود...**

```
delete [Name of the pointer];
```

عندما أكتب delete ما يحدث هو أن هذه الدالة delete تذهب الى عنوان المساحة المحجوزة في الذاكرة المخزن في البوينتر وتقوم بتحرير هذه المساحة.
عملية تحرير المساحة تعني... جعل المساحة المحجوزة من ال heap قابلة لإعادة الحجز لاحقا بعد حذف البيانات منها طبعا
(كلمة (البيانات) تعني القيمة المخزنة في المساحة)
الذاكرة التي أتحدث عنها في كل الشرح السابق اسمها ال heap سأتحدث عنها في الفكرة القادمة.

حسنا بعدما حررنا المساحة التي كانت محجوزة... عنوان هذه المساحة لا يزال مخزن في ال pointer ... اذا يجب أن أجعل البوينتر أن يتوقف عن الإشارة الى عنوان هذه المساحة المحررة (لا يشير إلى أي عنوان) **باستخدام الكود...**

```
[name of the pointer] = nullptr;
```

ترجمة nullptr الى العربية : بوينتر لا يشير الى شيئ (بوينتر فارغ).

ماهي ال Heap في الذاكرة ؟

هي مكان في الذاكرة يتم حجز منه مساحات التخزين الديناميكي مثل حجز مساحة مؤقتة في ال heap كما تعلمنا في آخر فكرة.

تعريف مساحة في ال heap على أنها مصفوفة :

نستخدم الكود...

```
int values;
int* pntr1;
cout << "Please enter how many values?\n";
cin >> values;
pntr1 = new int[values];
```

بهذه الطريقة أعرف مصفوفة ديناميكية يحدد عدد عناصرها المعرفة المستخدم وبهذه الطريقة لا أعرف عدد عناصر زيادة لا أحتاجه.

ولحذف ما حجزته من مساحة مصفوفة وتهيئة البوينتر :

```
delete [] pntr1;
pntr1 = nullptr;
```

عندما تستدعي الأمر [] delete فانت تطلب حذف كل عناصر المصفوفة مهما كان عددها ولهذا كتبنا الأقواس الفارغة (دلالة مهما كان عدد العناصر).

أشكال الخزانات في الذاكرة :

- أول خزان هو الخزان المخصص للخزانات : ال source code أو ال Instructions (الأوامر) وعادة تكون مساحة التخزين صغيرة.
- ثاني خزان هو الخزان المخصص للخزانات : المتغيرات ال static و ال global
- ثالث خزان هو الخزان المخصص للخزانات : المتغيرات المحلية بالنسبة للفنكشنز أو ال int main التي هي أيضا فنكشن أصلا أو الفنكشنز بحد ذاتهم و البوينترز أيضا
ولكن البوينترز هي مثل دفتر شيكات تستطيع استخدامها للولوج الى الخزان الرابع...
- الخزان الرابع هو الخزان المخصص للخزانات : ال heap الذي تحدثنا عنه سابقا.

للوصول الى عناصر مخزنة في المصفوفة :

هذه طريقة من طرق الوصول الى عنصر في الفيكتور باستخدام الدالة at .

```
vector <int> v{ 1,2,3,4 };
cout << "Num 1: " << v.at(0) << endl;
```

الوصول لكل عنصر من عناصر ال فيكتور باستخدام دالة ال for :

لدينا المثال الذي يشرح أننا نستطيع الوصول الى كل عنصر من عناصر الفيكتور ولكن بحيث لا نستطيع التعديل عليه (فقط استخدامه) **باستخدام الكود...**

```
for (const int& i : v)
        cout << i << endl;
```

لأننا كتبنا const أي أن العناصر داخل الفيكتور أصبح يعرفها البرنامج على أنها ثوابت ولا يمكن التعديل عليها (هذا الكلام محقق فقط داخل ال for loop)
أصبحت لا أستطيع التعديل عليها (التعديل على القيم) داخل البرنامج إنما فقط استخدامها.

ال Iterator :

ال iterator تعني المكرر باللغة العربية. برمجيا... شيئ يمر على كائن تلو الآخر.

هو يشبه المؤشر (pointer) الذي تعلمناه سابقا ولكن ال iterator يدعم المرور الذكي على كل الكائنات بمختلف أنواعها.
أي أنه pointer ولكن نسخة المطورة ديناميكيا. ونعم... يخزن عناوين.

لتعريف iterator بمسمى...

```
vector <int>::iterator itr;
```

أسميته itr.
لماذا استخدمنا :: عند تعريفه؟ لأننا نحاول الوصول الى أداة موجودة داخل التعريف vector <int> الذي طبعا يكون معرف.

الآن سنعطي مثال على أحد استخدامات ال vector interaror ...

```
vector <int> v{ 1,2,3,4 };
vector <int>::iterator itr;

for (itr = v.begin();itr != v.end();itr++)
{
        cout << "first member using an Interator: " << *itr << endl;
}
return 0;
```

ملاحظة: الكود v.end() لن يضع المؤشر عند آخر عنصر. بل العنصر الغير موجود الذي هو بعد الأخير. بهذا عند وصول المؤشر الى ما بعد الأخير تنتهي ال forloop كما نلاحظ في المثال السابق وهذا المطلوب لأننا نريد أن آخر أمر ينفذ يكون على العنصر الأخير.

نستطيع كتابة سطر ال for loop بالشكل...

```
for (itr = v.begin();itr<= v.end()-1;itr++)
```

ولكن لأن interator ال vector يدعم العمليات الحسابية مثل – أو + يتم الأمر بشكل سليم ولكن لاحقا هناك أنواع لا تدعم العمليات الحسابية لذلك يفضل استخدام الطريقة في كتابة سطر for المعروضة في المثال السابق.

طباعة عنوان قيمة مشار اليها من قبل interator :

لا أستطيع طباعة عنوان العنصر المؤشر عليه حاليا لأن عمليات الطباعة لا تدعم طباعة عنوان قيمة مخزنة داخل interator فإذا... يجب أن أطلب قيمة العنصر ثم أطلب التحديد على عنوانه (فك ال interator) باستخدام الإشارة & كما في الكود الآتي...

```
vector <int> v{ 1,2,3,4 };
vector <int>::iterator itr;

for (itr = v.begin();itr != v.end();itr++)
{
        cout << "first address using an Interator: " << &(*itr) << endl;
}
```

نوع الخطأ crash وكيف التعامل معه :

ال crash هو خطأ يحدث أثناء فترة التشغيل (run time) يسبب توقف البرنامج عن العمل.
ويحدث بسبب أمر قمت بطلب تنفيذه ولكن لم يستطع ال كومبايلر تنفيذه لأنه غير ممكن التنفيذ.

الخيار الأول... (وهو الأصح) الذهاب الى الكود الذي أشك أنه خطأ وأقوم بالتعديل عليه إن أمكن تعديله.
الخيار الثاني... (لا يستخدم إلا عند الضرورة القصوى لأنه يسبب بطأ في البرنامج) وهو إستخدام كودين يقومان بالتحقق من الكود الذي أشك أنه قد يسبب أخطاء للتعامل معه.

مثال عن برنامج يسبب crash والتعامل معه باستخدام الكودين المتحدث عنهم...

برنامج يسبب crash :

```
vector <int> v{ 1,2,3,4 };
cout << v.at(4) << endl;
```

طريقة التعامل معه :

```
vector <int> v{ 1,2,3,4 };
try
{
	cout << v.at(4);
}
catch (...)
{
	cout << "Not a supported line of code\n";
}
```

المعنى : اذا سبب الكود المكتوب داخل body ال try بعد تجريبه... crash . تجاهله ونفذ الكود في body ال catch .

دوال ال string object :

ما هو ال string object ؟... عندما أعرف سترينغ بعد إستدعاء مكتبته فهذا اسمه string object هذا السترينغ له دوال موجودة داخله يمكن استخدامها معه...

ملاحظة على أحد الدوال...

```
.insert(0, " Hi ");
```

تضيف خانات في الموضع المختار داخل الدالة. حيث يضاف الكلام خلف عنوان الخانة المختارة ...

ملاحظة على أحد الدوال...

```
.find(" Hi ");
```

الدالة تبحث عن السلسلة النصية المذكورة وتعيد عنوان أول خانة في السلسلة.

ملاحظة على أحد الدوال ضمن مثال...

```
if (sentence.find("hi") == sentence.npos)
{
        cout << "hi is not found!\n";
}
```

الدالة npos تخزن قيمة. هذه القيمة دلالة " غير موجود " .
في الدالة find. ... اذا رجعت لها قيمة تساوي القيمة المخزنة في npos دلالة " غير موجود " تنفذ الكود الموجود في ال Body تبع ال if . (دلالة رجعت لل find قيمة نفس قيمة npos التي قيمتها تدل على عدم الوجود) .

الدالة npos تعني بالانجليزية... no position . بالعربية... غير موجود.

دوال ال char :

الدالة toupper و tolower ترجعان رقم على حسب طاولة ال askii اذا toupper فهي ترجع رقم في ال askii يمثل الحرف الكبير من الحرف المختار وعند تخزين الرقم ال askii في متغير char فإن الرقم سوف يتحول إلى حرف وبهكذا نحول حرف صغير إلى كبير وإذا كان الحرف أصلا كبير فيبقى كبير.
ونفس الأمر في الدالة tolower ولكن مع عكس الكلمات (كلمة صغير تصبح كبير وكلمة كبير تصبح صغير على حساب ورودهم في السياق السابق) .

الكلام الذي سأكتبه الآن بين قوسين سوف يطبق على كل الدوال التي سوف أكتبها الآن (اذا رجعت الدالة قيمة 0 معناها false وأي رقم غير ال 0 معناها true)

الدالة isupper و islower ترجعان قيمة اما 0 أو أي رقم ثاني .

الدالة isdigit ترجع booleon وهي تتحقق اذا كان الخانة هي رقم من 0 حتى 9 . أي شيئ غير ذلك ترجع 0 .

الدالة ispunct ترجع booleon أي شيئ غير الخانات المميزة ترجع 0 .

كود تعريف ملف للكتابة عليه :

عندما نكتب كود تعريف class ملف **كالآتي...**

```
fstream Myfile;
```

ما حصل أنني عرفت class اسمه Myfile من النوع fstream .
والآن مربوط بالقرص الصلب ولكنه غير موصول الى ملف معين بعد .

ولدينا أمر فتح ملف...

```
Myfile.open("Myfile.txt", ios::out);
```

الأمر ios هو كلاس موجود في مكتبة الإدخال و الإخراج <iostream> هذا الكلاس له عدة ثوابت تحدد نوع الملف مثل الثابت الي يمثله الكود out والذي يحدد نوع الملف... " كتابة / write mode " .
وهناك الكثير من الثوابت الأخرى... التي تحدد مهمة الملف عند الفتح.

الأمر Myfile.close() مهم جدا لإغلاق الملف لأن لو أحد حاول يفتح الملف (نتعلم ذلك في مرحلة متقدمة) سوف يخبره أن الملف is in use ولن يستطيع الولوج للملف .

أيضا... ال out تقوم بفتح ملف بالاسم الذي كتبته إذا لم يكن موجود هناك ملف بهذا الاسم .

وهناك ثابت يقوم بتحديد الملف على أنه للإلحاق (يكتب في نهاية الملف ما أقول له أن يكتب وليس استبدال المكتوب في الملف) .

وهذا الثابت هو...

```
ios::app
```

app دلالة append (الإلحاق) .

يفضل عدم حذف ثابت الكتابة على الملف عند طلب ثابت الإلحاق في الملف... لأن ثابت الكتابة على الملف له مهمة ثانية وهي أنه يقوم بفتح ملف بالاسم الذي كتبته إذا لم يكن موجود هناك ملف بهذا الاسم .

ليصبح الكود في حال طلب ثابت الإلحاق بالشكل...

```
Myfile.open("Myfile.txt", ios::out | ios::app);
```

طبعا ال compiler يعطي أولوية التنفيذ لل append عند تضارب المهام ... (كتابة أم إلحاق ؟)

ثابت فتح ملف للإدخال... الثابت هو...

```
ios::in
```

مثال عليه باستخدام الكود...

```
fstream Myfile;
Myfile.open("Myfile.txt", ios::in);

if (Myfile.is_open());
{
        string lineofcode;

        while (getline(Myfile, lineofcode))
        {
                cout << lineofcode << endl;
        }
        Myfile.close();
}
```

سطر الwhile ترجمته : إذا وصلتك نتيجة true من عملية ال getline فقم بتنفيذ ال Body تبع ال while .

تصل نتيجة true إذا وجد سطر لتخزينه (عند محاولة تخزين السطر الجاري التأشير عليه) داخل ال lineofcode

و تصل نتيجة false عند محاولة تخزين سطر ولكن لا يوجد سطر لتخزينه.

طبعا قبل كل عملية تنفيذ body ... يخزن السطر المؤشر عليه في الملف داخل ال lineofcode وثم ينفذ ال body .
وفي العملية التالية ينتقل المؤشر الى السطر التالي لتطبيق الأوامر باستخدامه.

كتابة أوامر الزمن واستدعاء مكتبة الزمن في c++ :

الدوال التي نستخدمها قد تؤثر على البرنامج بطريقة سلبية... لذلك يعطينا الكومبايلر تحذير عند تسغيل البرنامج... وبما أننا متأكدين أننا نستخدم هذه الدوال بشكل سليم... نكتب كود ليتجاهل هذا التحذير... ويعمل البرنامج .
هذا الكود هو...

```
#pragma warning (disable : 4996)
```

المكتبة التي استخدمناها #include<ctime> موروثة من لغة C .

الكود : time_t [name of the variable] يستخدم لتعريف متغير لتخزين الوقت بالثواني منذ الزمن 1970 .
وعندما أكملنا الكود لنقول... time(0)= معناها أن نقطة النهاية لعدد الثواني هو الثانية التي ينفذ بها الكود.

هذا الكود ctime(&t) يحول الرقم من الثواني الى مصفوفة من أحرف تمثل الوقت بشكل سلسلة نصية من مصفوفة أحرف.

وعندما أقول... char* st = ctime(&t); فأنا أعرف بوينتر يشير إلى أول عنصر في المصفوفة النصية (أول حرف) و الآن يستطيع الكوبايلر الوصول لكل عناوين الأحرف في المصفوفة من خلال المؤشر فقط بالزيادة +1 لينتقل الى العنوان التالي .

هذا الكود... tm* gmtm = gmtime(&t); يعرف متغير لتخزين عنوان بنية زمن (قالب زمن)
ونعطي المتغير لدالة التحويل لتحويله الى غرنتش بالنسبة لعنوان الزمن المخزن في t وليس بالنسبة لزمن تنفيذ الكود هذا.

الكود... st = asctime(gmtm); يخزن قالب الزمن كسلسلة نصية ويخزنها في المتغير st التي سابقا كان مخزن بها الزمن عند التنفيذ كود إعطاء الزمن الحالي أما الآن فخزنا فيها نفس الزمن بالنسبة لغرينيتش.

مصطلحات هامة من الكورس 6

Debugging إكتشاف الأخطاء وتصحيحها .

Syntax error خطأ في قواعد الكتابة .

Logical error خطأ منطقي .

RunTime error خطأ أثناء التشغيل .

Breakpoint نقطة تحدد مكان بدأ ال debugging .

Autos نافذة إظهار قيم المتغيرات .

Quick Watch نافذة مراقبة المتغيرات السريعة .

Revaluate التعديل المؤقت على المعادلات وقيمها .

Step into للنزول سطر جديد عند ال debugging .

Step out للخروج من داخل دالة عند ال debugging .

Step over لتخطي الدخول في دالة عند ال debugging .

Lib يرمز الى library (مكتبة) .

Ternary Operator إختصار كتابة الكود .

Ranged loop الحلقة المحدودة .

Bitwise مقارنة كل خانة binary مع أخرى (أحاديات المعامل) .

Declaration تعريف أولي .

Definition تعريف فعلي .

Parameters المتغيرات التي تمرر للدالة .

Arguments القيم الفعلية للمتغيرات التي تمرر للدالة .

Function overloading عمل أكثر من دالة تحت نفس الإسم .

Polymorphism تعدد الأشكال .

Call Stack / Call Hierarchy حزمة الإستدعاء .

Active Frame الإطار الفعال الفعلي (الدالة الجاري تنفيذها حاليا) .

Push إضافة عنصر إلى ال call stack .

Pop إزالة عنصر من ال call stack .

Recursion دالة تستدعي نفسها عدد من المرات أنت تحدده (العودة الذاتية / الإستدعاء) .

Stack overflow تجاوز الحد المسموح به للإطارات داخل ال Call stack .

Static variable متغير فترة حياته لا تنتهي بانتهاء فترة حياة الدالة .

Auto variable متغير يعرف نفسه بنفسه على حسب القيم التي تخزنها به .

Register variable متغير مخزن في الذاكرة المسماة register (أقرب ذاكرة لل cpu) .

Expression برمجيا : عملية حسابية .

Print f بمعنى طباعة منسقة (print formated) .

Tow dimensional array مصفوفة ثنائية الأبعاد .

Vector مصفوفة ديناميكية .

Call by value تمرير نسخة من المتغيرللدالة المستدعاه .

Call by ref الإشارة الى المتغير الممرر للدالة المستدعاه .

Reference مرجع .

Pointer مؤشر .

Dereferencing عملية الوصول إلى قيمة المتغير المخزن عنوانه في ال pointer .

Generic pointer متغير عام .

Body (جسم) ما يكتب بين القوسين ال curly لأي دالة .